

Collaborative filtering

Making recommendations

Predicting movie ratings

User rates movies using one to five stars

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)
Love at last	5	5	0	0
Romance forever	5	?	?	0
Cute puppies of love	?	4	0	?
Nonstop car chases	0	0	5	4
Swords vs. karate	0	0	5	?

Handwritten notes: "zero" above Alice(1) for Love at last; "4" above Bob(2) for Cute puppies of love.

★				
★	★			
★	★	★		
★	★	★	★	
★	★	★	★	★

$n_u = 4$ (no. of users)
 $n_m = 5$ (no. of movies)
 $r(i,j) = 1$ if user j has rated movie i
 $y^{(i,j)}$ = rating given by user j to movie i (defined only if $r(i,j)=1$)

$r(1,1) = 1$
 $r(3,1) = 0$
 $y^{(3,2)} = 4$

→ if a people hasn't voted yet, the rating of that movie will be "?"

Using per-item features

What if we have features of the movies?

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)	x_1 (romance)	x_2 (action)
Love at last	5	5	0	0	0.9	0
Romance forever	5	?	?	0	1.0	0.01
Cute puppies of love	?	4	0	?	0.99	0
Nonstop car chases	0	0	5	4	0.1	1.0
Swords vs. karate	0	0	5	?	0	0.9

Handwritten notes: "5" above Alice(1) for Love at last; "4" above Bob(2) for Cute puppies of love.

$n_u = 4$
 $n_m = 5$
 $n = 2$

$x^{(1)} = \begin{bmatrix} 0.9 \\ 0 \end{bmatrix}$
 $x^{(3)} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix}$

For user 1: Predict rating for movie i as: $w^{(1)} \cdot x^{(i)} + b^{(1)}$ (just linear regression)

$w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$ $b^{(1)} = 0$ $x^{(3)} = \begin{bmatrix} 0.99 \\ 0 \end{bmatrix}$
 $w^{(1)} \cdot x^{(3)} + b^{(1)} = 4.95$

→ For user j : Predict user j 's rating for movie i as $w^{(j)} \cdot x^{(i)} + b^{(j)}$

What if we additionally have features of the movies? So here I've added two features X_1 and X_2 , that tell us how much each of these is a romance movie, and how much each of these is an action movie. So for example Love at Last is a very romantic movie, so this feature takes on 0.9, but it's not at all an action movie. So this feature takes on 0. But it turns out Nonstop Car chases has just a little bit of romance in it.

So it's 0.1, but it has a ton of action. So that feature takes on the value of 1.0.

Cost function

So let's take a look at how we can formulate the cost function for this algorithm

Cost function

Notation:

- $r(i,j) = 1$ if user j has rated movie i (0 otherwise)
- $y^{(i,j)}$ = rating given by user j on movie i (if defined)
- $w^{(j)}, b^{(j)}$ = parameters for user j
- $x^{(i)}$ = feature vector for movie i

For user j and movie i , predict rating: $\underbrace{w^{(j)} \cdot x^{(i)}} + b^{(j)}$

→ $m^{(j)}$ = no. of movies rated by user j

To learn $\underline{w^{(j)}}, \underline{b^{(j)}}$

$$\min_{w^{(j)}, b^{(j)}} J(w^{(j)}, b^{(j)}) = \frac{1}{2m^{(j)}} \sum_{i:r(i,j)=1} (\underbrace{w^{(j)} \cdot x^{(i)}} + \underbrace{b^{(j)}} - y^{(i,j)})^2 + \frac{\lambda}{2m^{(j)}} \sum_{k=1}^n \underbrace{(w_k^{(j)})^2}_{\text{number of features}}$$

it turns out that for recommender systems it would be convenient to actually eliminate this division by $m^{(j)}$ term, $m^{(j)}$ is just a constant in this expression.

And so, even if you take it out, you should end up with the same value of w and b . $m^{(j)}$ is just a constant in this expression. And so, even if you take it out, you should end up with the same value of w and b .

$i : r(i, j) = 1$: Means all i where $r(i, j) = 1$

Cost function

To learn parameters $w^{(j)}, b^{(j)}$ for user j :

$$J(w^{(j)}, b^{(j)}) = \frac{1}{2} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (w_k^{(j)})^2$$

To learn parameters $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$ for all users :

$$J(w^{(1)}, \dots, w^{(n_u)}, b^{(1)}, \dots, b^{(n_u)}) = \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} \underbrace{(w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2}_{f(x)} + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Collaborative filtering algorithm

Problem motivation

Problem motivation

Movie	Alice (1)	Bob (2)	Carol (3)	Dave (4)	x_1 (romance)	x_2 (action)
Love at last	5	5	0	0	?	?
Romance forever	5	?	?	0	?	?
Cute puppies of love	?	4	0	?	?	?
Nonstop car chases	0	0	5	4	?	?
Swords vs. karate	0	0	5	?	?	?

$$w^{(1)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(2)} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}, w^{(3)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}, w^{(4)} = \begin{bmatrix} 0 \\ 5 \end{bmatrix}$$

$$b^{(1)} = 0, b^{(2)} = 0, b^{(3)} = 0, b^{(4)} = 0$$

using $w^{(j)} \cdot x^{(i)} + b^{(j)}$

$$\begin{aligned} w^{(1)} \cdot x^{(1)} &\approx 5 \\ w^{(2)} \cdot x^{(1)} &\approx 5 \\ w^{(3)} \cdot x^{(1)} &\approx 0 \\ w^{(4)} \cdot x^{(1)} &\approx 0 \end{aligned} \rightarrow x^{(1)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

j : j' th user

i : i' th movie

In the last video, you saw how if you have features for each movie, such as features x_1 and x_2 that tell you how much is this a romance movie and how

much is this an action movie. Then you can use basically linear regression to learn to predict movie ratings. But what if you don't have those features, x_1 and x_2 ?

Let's take a look at how you can learn or come up with those features x_1 and x_2 from the data. Here's the data that we had before. But what if instead of having these numbers for x_1 and x_2 , we didn't know in advance what the values of the features x_1 and x_2 were? I'm going to replace them with question marks (?) over here. Now, just for the purposes of illustration, let's say we had somehow already learned parameters for the four users.

Let's say that we learned parameters $w^{(1)} = [5 \ 0]$ and $b^{(1)} = 0$, for user one. $w^{(2)}$ is also $[5, 0]$, $b^{(2)} = 0$, We'll worry later about how we might have come up with these parameters, w and b . But let's say we have them already. As a reminder, to predict user j 's rating on movie i , we're going to use $w^{(j)} \cdot x^{(i)} + b^{(j)}$. To simplify this example, all the values of b are actually equal to 0. Just to reduce a little bit of writing, I'm going to ignore b for the rest of this example.

Let's take a look at how we can try to guess what might be reasonable features for movie one. If these are the parameters you have on the left, then given that Alice rated movie one 5 star, we should have that $w^{(1)} \cdot x^{(1)} = 5$ and $w^{(2)} \cdot x^{(1)} = 5$, $w^{(3)} \cdot x^{(1)} = 0$, $w^{(4)} \cdot x^{(1)} = 0$. The question is, given these values for w that we have up here, what choice for x_1 will cause these values to be right?

Well, one possible choice would be if the features for that first movie, were 1 and 0 in which case, $w^1 \cdot x^1$ will be equal to 5, $w^2 \cdot x^1$ will be equal to 5 and similarly, w^3 or w^4 dot product with this feature vector x_1 would be equal to 0.

What we have is that if you have the parameters for all four users here, and if you have four ratings in this example that you want to try to match, you can take a reasonable guess at what lists a feature vector x_1 for movie one that would make good predictions for these four ratings up on top.

Similarly, if you have these parameter vectors, you can also try to come up with a feature vector x_2 for the second movie, feature vector x_3 for the third movie, and so on to try to make the algorithm's predictions on these additional movies close to what was actually the ratings given by the users.

Let's come up with a cost function for actually learning the values of x_1 and x_2 . By the way, notice that this works only because we have parameters for four users. That's what allows us to try to guess appropriate features, x_1 . This is why in a typical linear regression application if you had just a single user, you don't actually have enough information to figure out what would be the features, x_1 and x_2 , which is why in the linear regression contexts that you saw in course 1, you can't come up with features x_1 and x_2 from scratch. But in collaborative filtering, is because you have ratings from multiple users of the same item with the same movie. That's what makes it possible to try to guess what are possible values for these features.

Cost function

Given $w^{(1)}, b^{(1)}, w^{(2)}, b^{(2)}, \dots, w^{(n_u)}, b^{(n_u)}$

to learn $x^{(i)}$:

$$J(x^{(i)}) = \frac{1}{2} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{k=1}^n (x_k^{(i)})^2$$

→ To learn $x^{(1)}, x^{(2)}, \dots, x^{(n_m)}$:

$$J(x^{(1)}, x^{(2)}, \dots, x^{(n_m)}) = \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Collaborative filtering

Cost function to learn $w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}$:

$$\min_{w^{(1)}, b^{(1)}, \dots, w^{(n_u)}, b^{(n_u)}} \frac{1}{2} \sum_{j=1}^{n_u} \sum_{i:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Cost function to learn $x^{(1)}, \dots, x^{(n_m)}$:

$$\min_{x^{(1)}, \dots, x^{(n_m)}} \frac{1}{2} \sum_{i=1}^{n_m} \sum_{j:r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

Put them together:

$$\min_{\substack{w^{(1)}, \dots, w^{(n_u)} \\ b^{(1)}, \dots, b^{(n_u)} \\ x^{(1)}, \dots, x^{(n_m)}}} J(w, b, x) = \frac{1}{2} \sum_{(i,j):r(i,j)=1} (w^{(j)} \cdot x^{(i)} + b^{(j)} - y^{(i,j)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

	j=1	j=2	j=3
	Alice	Bob	Carol
i = 1 Movie1	5	5	?
i = 2 Movie2	?	2	3

How do you minimize this cost function as a function of w , b , and x ?

Gradient Descent

collaborative filtering

Linear regression (course 1)

repeat {

$$\cancel{w_i = w_i - \alpha \frac{\partial}{\partial w_i} J(w, b)} \quad w_i^{(j)} = w_i^{(j)} - \alpha \frac{\partial}{\partial w_i^{(j)}} J(w, b, x)$$

$$\cancel{b = b - \alpha \frac{\partial}{\partial b} J(w, b)} \quad b^{(j)} = b^{(j)} - \alpha \frac{\partial}{\partial b^{(j)}} J(w, b, x)$$

$$x_k^{(i)} = x_k^{(i)} - \alpha \frac{\partial}{\partial x_k^{(i)}} J(w, b, x)$$

}

parameters w, b, x x is also a parameter

The algorithm we derived is called collaborative filtering, and the name collaborative filtering refers to the sense that because multiple users have rated the same movie collaboratively, given you a sense of what this movie maybe like, that allows you to guess what are appropriate features for that movie, and this in turn allows you to predict how other users that haven't yet rated that same movie may decide to rate it in the future.

Binary labels: favs, likes and clicks

Binary labels

Movie	Alice(1)	Bob(2)	Carol(3)	Dave(4)
Love at last	1	1	0	0
Romance forever	1	? ←	? ←	0
Cute puppies of love	? ←	1	0	? ←
Nonstop car chases	0	0	1	1
Swords vs. karate	0	0	1	? ←

1
0
?

Here's an example of a collaborative filtering data set with binary labels. A one denotes that the user liked or engaged with a particular movie. So label one could mean that Alice watched the movie Love at last all the way to the end and watch romance forever all the way to the end.

But after playing a few minutes of nonstop car chases decided to stop the video and move on. Or it could mean that she explicitly hit like or favorite on an app to indicate that she liked these movies.

But after checking out nonstop car chasers and swords versus karate did not hit like. And the question mark usually means the user has not yet seen the item and so they weren't in a position to decide whether or not to hit like or favorite on that particular item. So the question is how can we take the collaborative filtering algorithm that you saw in the last video and get it to work on this dataset. And by predicting how likely Alice, Bob carol and Dave are to like the items that they have not yet rated, we can then decide how much we should recommend these items to them. There are many ways of defining what is the label one and what is the label zero, and what is the label question mark in collaborative filtering with binary labels.

Example applications

- 1. Did user j purchase an item after being shown? 1, 0, ?
- 2. Did user j fav/like an item? 1, 0, ?
- 3. Did user j spend at least 30sec with an item? 1, 0, ?
- 4. Did user j click on an item? 1, 0, ?

Meaning of ratings:

- 1 - engaged after being shown item
 - 0 - did not engage after being shown item
 - ? - item not yet shown
-

From regression to binary classification

- Previously:
- Predict $y^{(i,j)}$ as $\underline{w^{(j)} \cdot x^{(i)} + b^{(j)}}$
- For binary labels:
 Predict that the probability of $y^{(i,j)} = 1$
 is given by $\underline{g(w^{(j)} \cdot x^{(i)} + b^{(j)})}$
 where $\underline{g(z) = \frac{1}{1+e^{-z}}}$

Cost function for binary application

Previous cost function:

$$\frac{1}{2} \sum_{(i,j): r(i,j)=1} (\underbrace{w^{(j)} \cdot x^{(i)} + b^{(j)}}_{f(x)}) - y^{(i,j)} \Big)^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2$$

Loss for binary labels $y^{(i,j)}$: $f_{(w,b,x)}(x) = g(w^{(j)} \cdot x^{(i)} + b^{(j)})$

$$L(f_{(w,b,x)}(x), y^{(i,j)}) = -y^{(i,j)} \log(f_{(w,b,x)}(x)) - (1 - y^{(i,j)}) \log(1 - f_{(w,b,x)}(x)) \quad \leftarrow \text{Loss for single example}$$

$$J(w, b, x) = \sum_{(i,j): r(i,j)=1} L(\underbrace{f_{(w,b,x)}(x)}_{g(w^{(j)} \cdot x^{(i)} + b^{(j)})}, y^{(i,j)}) \quad \leftarrow \text{cost for all examples}$$